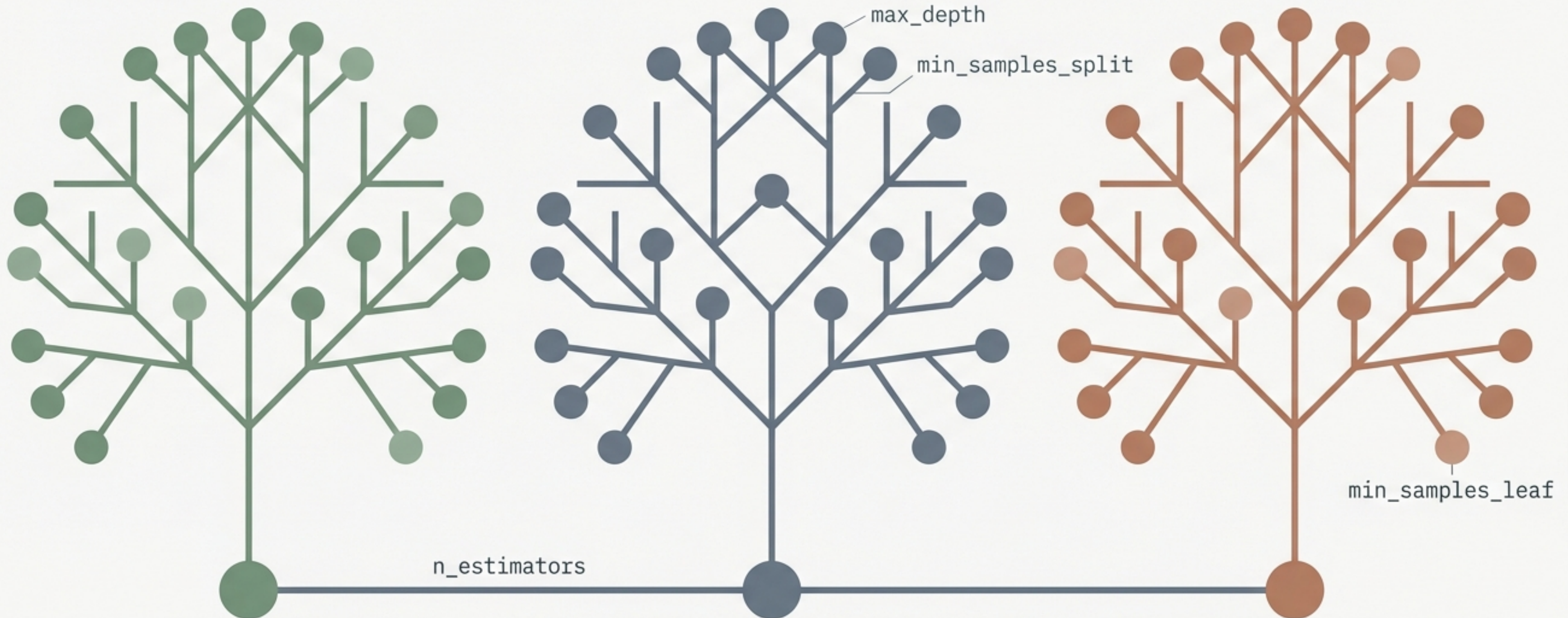


Mastering Random Forest Hyperparameters

A visual playbook for tuning classifiers and regressors, managing algorithmic flexibility, and building better machine learning models.



Taming algorithmic flexibility

Random Forests are extraordinarily flexible algorithms. While this flexibility makes them powerful, it also makes them prone to overfitting if not properly tuned. The goal of hyperparameter tuning is to smooth out decision boundaries and stop the model from memorising noise.



Overfitted



Optimised

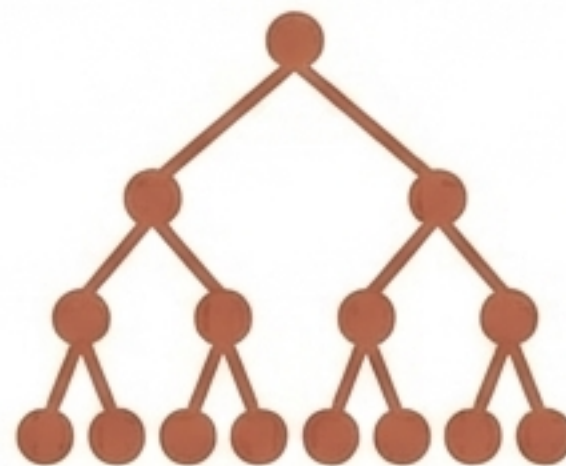
The three-part architecture of tuning

To simplify tuning, we divide the parameters into three distinct operational categories based on their function in the algorithm.



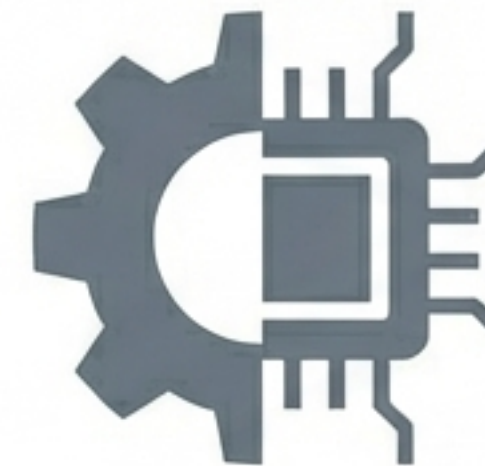
1. The Forest

Parameters governing the macroscopic Random Forest.



2. The Trees

Parameters controlling the microscopic, internal decision trees.



3. The System

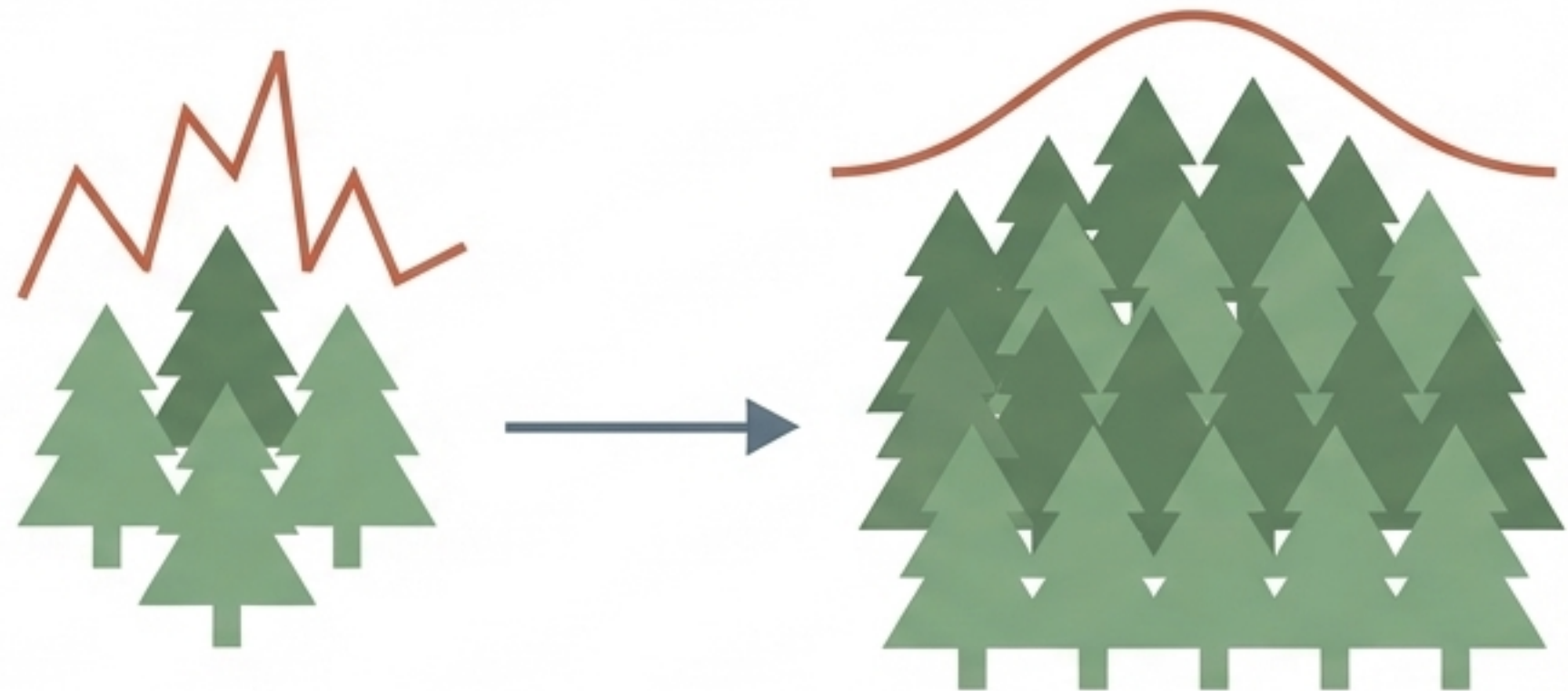
General machine learning parameters controlling the computing environment.

n_estimators

Definition: The total number of independent decision trees that make up your Random Forest.

Default Value: 100.

Impact: Increasing the number of trees smooths out the decision boundary and drastically reduces overfitting. It diminishes erratic, overfitted zones in the data space.



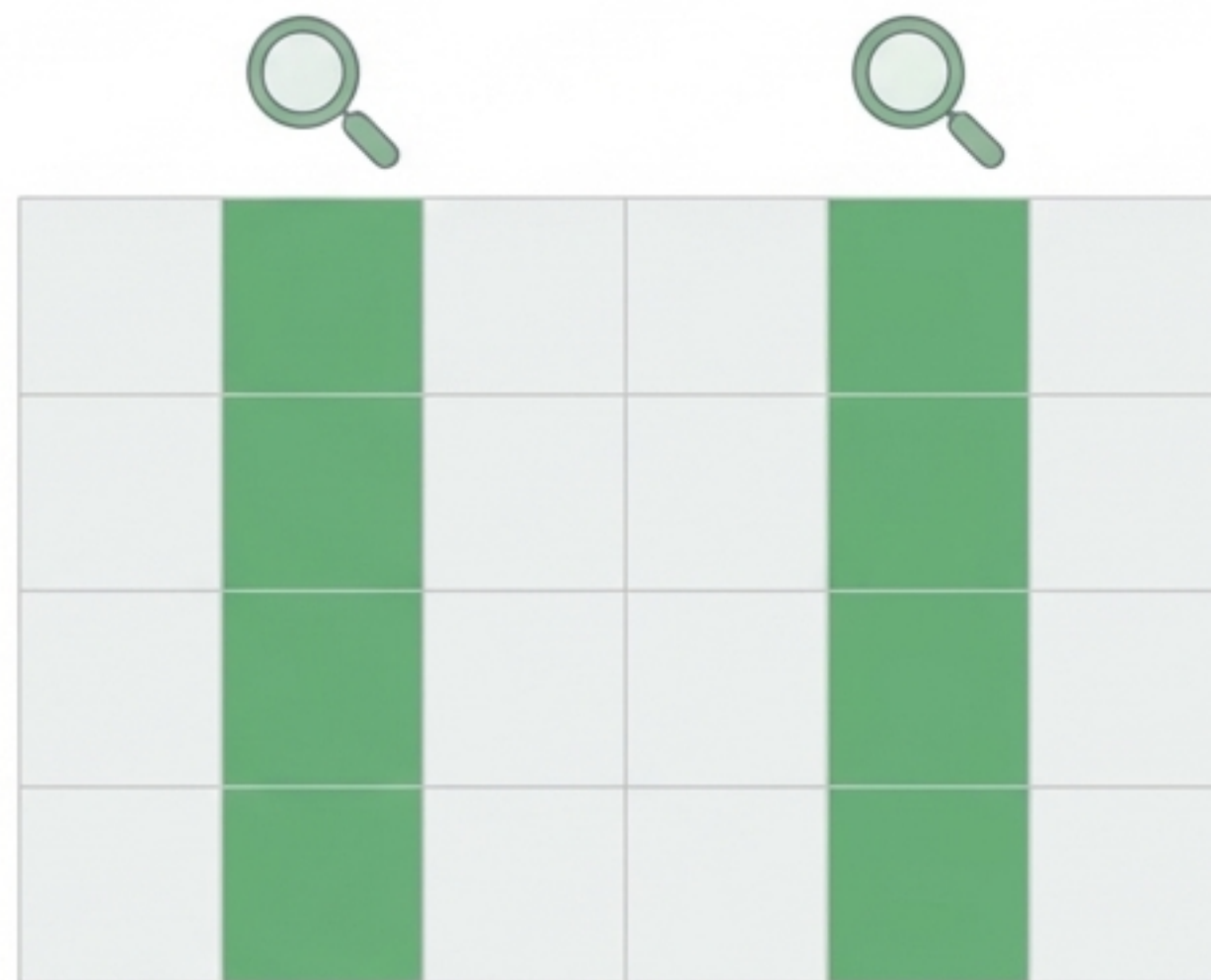
max_features

Definition: The number of features (columns) the algorithm considers when looking for the best split.

Options:

- `Auto` / `Sqrt` (Takes the square root of total features)
- `Log2` (Log base 2 of total features)
- `Float` (A precise percentage, e.g., 0.5 for 50%)
- `Integer` (A strict numeric limit)

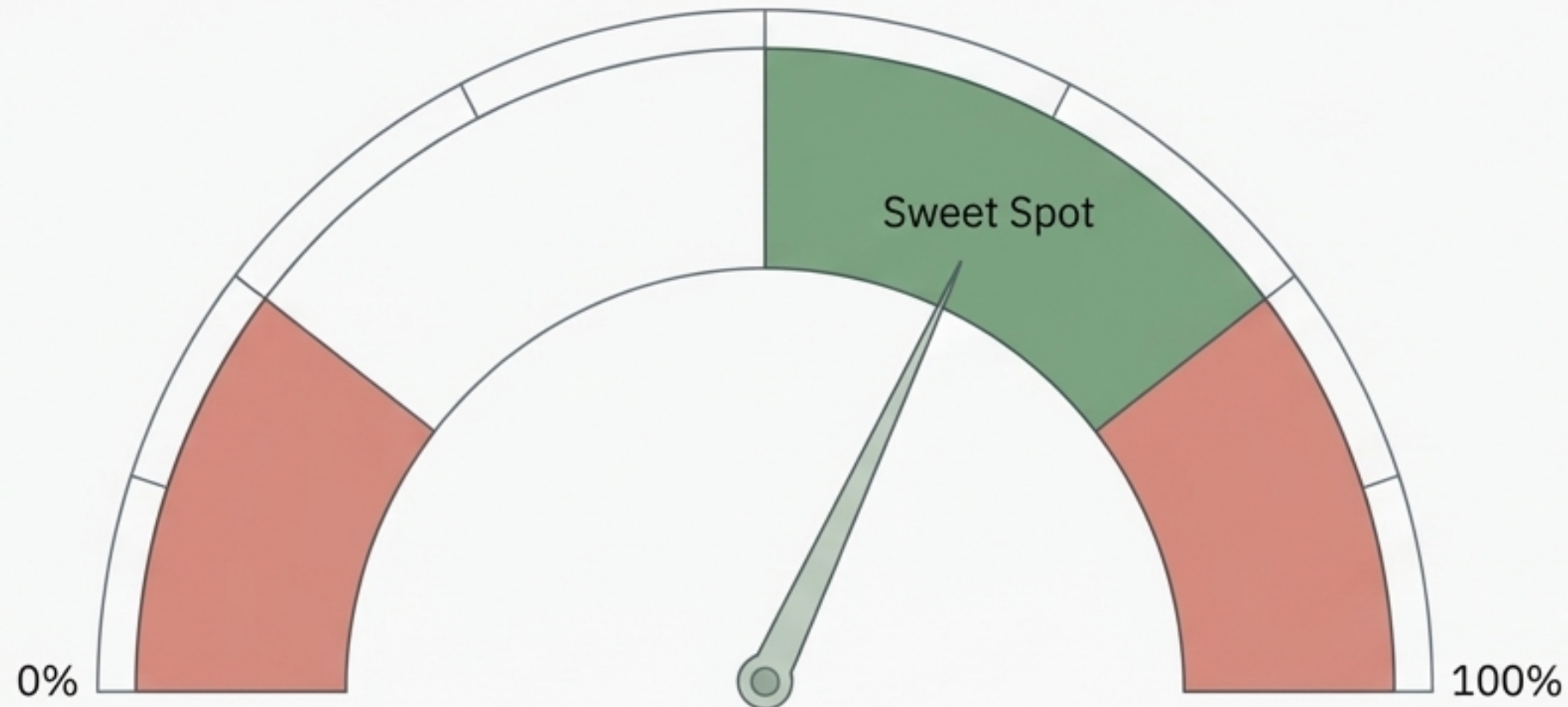
Impact: Restricting features introduces randomness and prevents dominant columns from overshadowing subtle data patterns.



max_samples

Definition: The specific number or percentage of rows from the dataset provided to each individual tree.

Pro Tip: For the most optimum results, aim for the 50% to 75% range. Pushing this value too high (approaching 100%) or dropping it too low drastically degrades model accuracy.



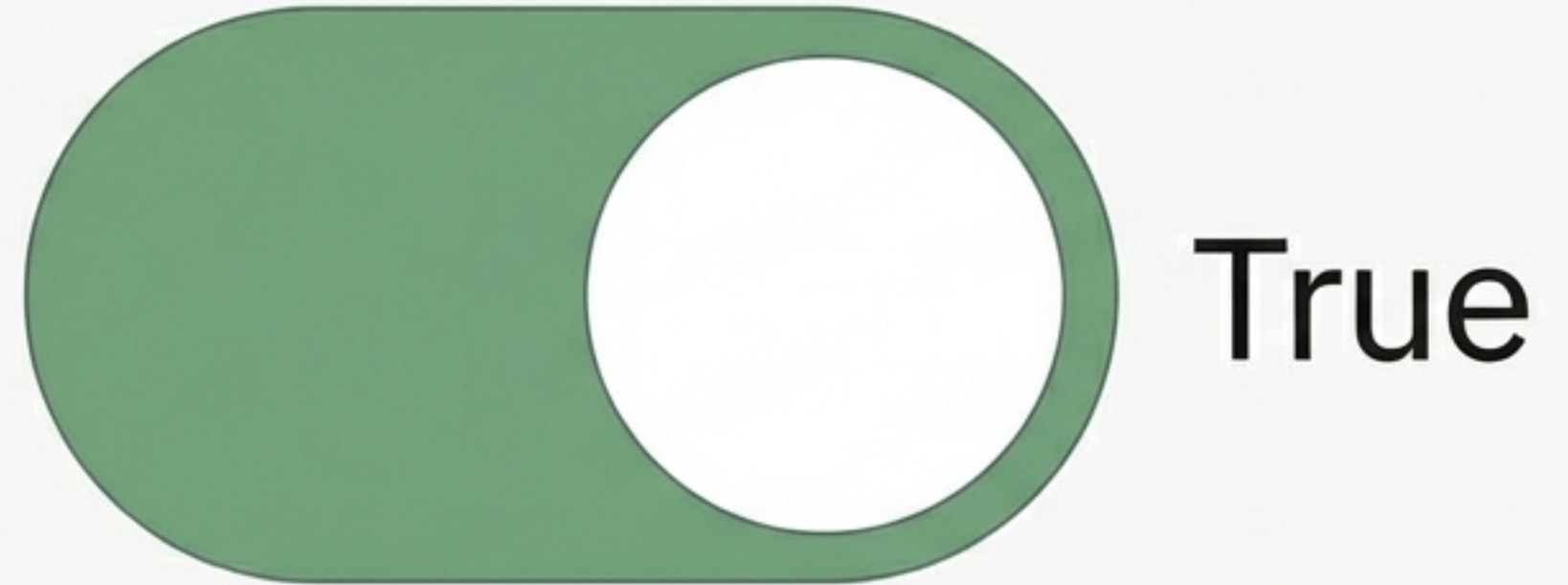
bootstrap

Definition: Determines whether data sampling for each tree is done with replacement.

Options:

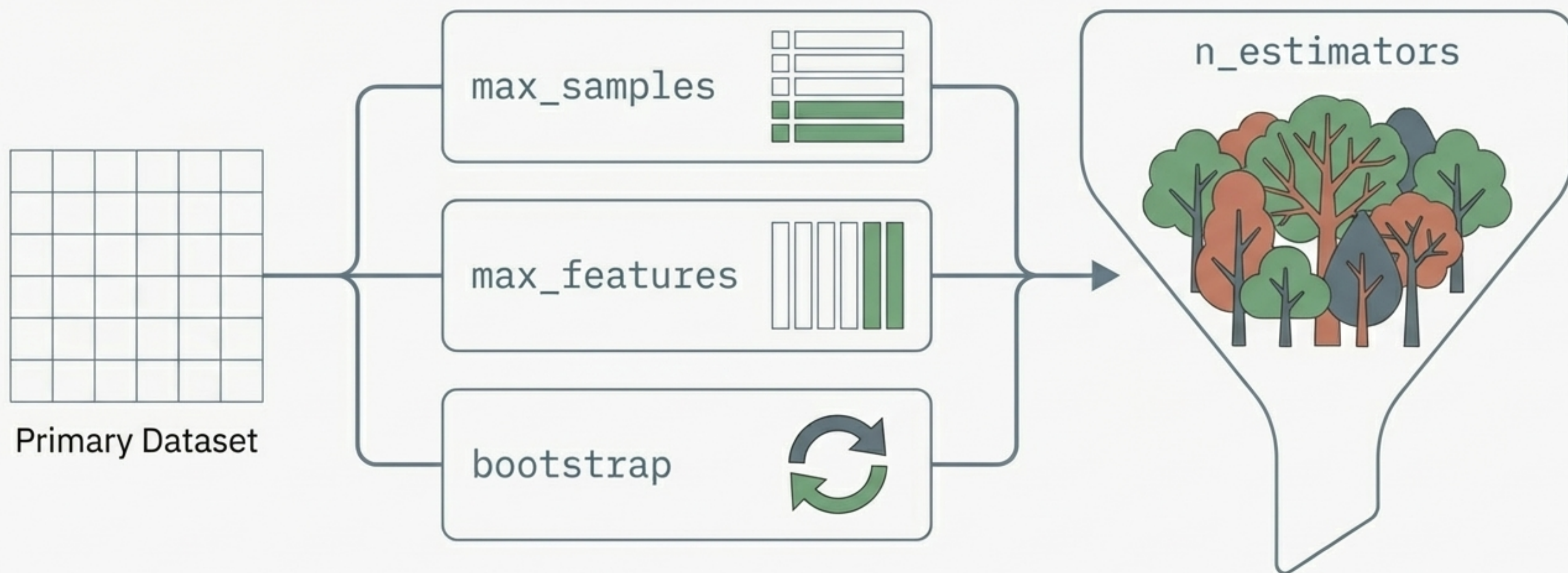
True or False. (Default is True).

Impact: While you can toggle this this to False (sampling without replacement), general consensus and real-world testing show that maintaining the True default is overwhelmingly preferred and rarely requires changing.



Orchestrating controlled randomness

These four parameters do not work in isolation. Together, they dictate exactly how much unique data and structural variety each tree receives.

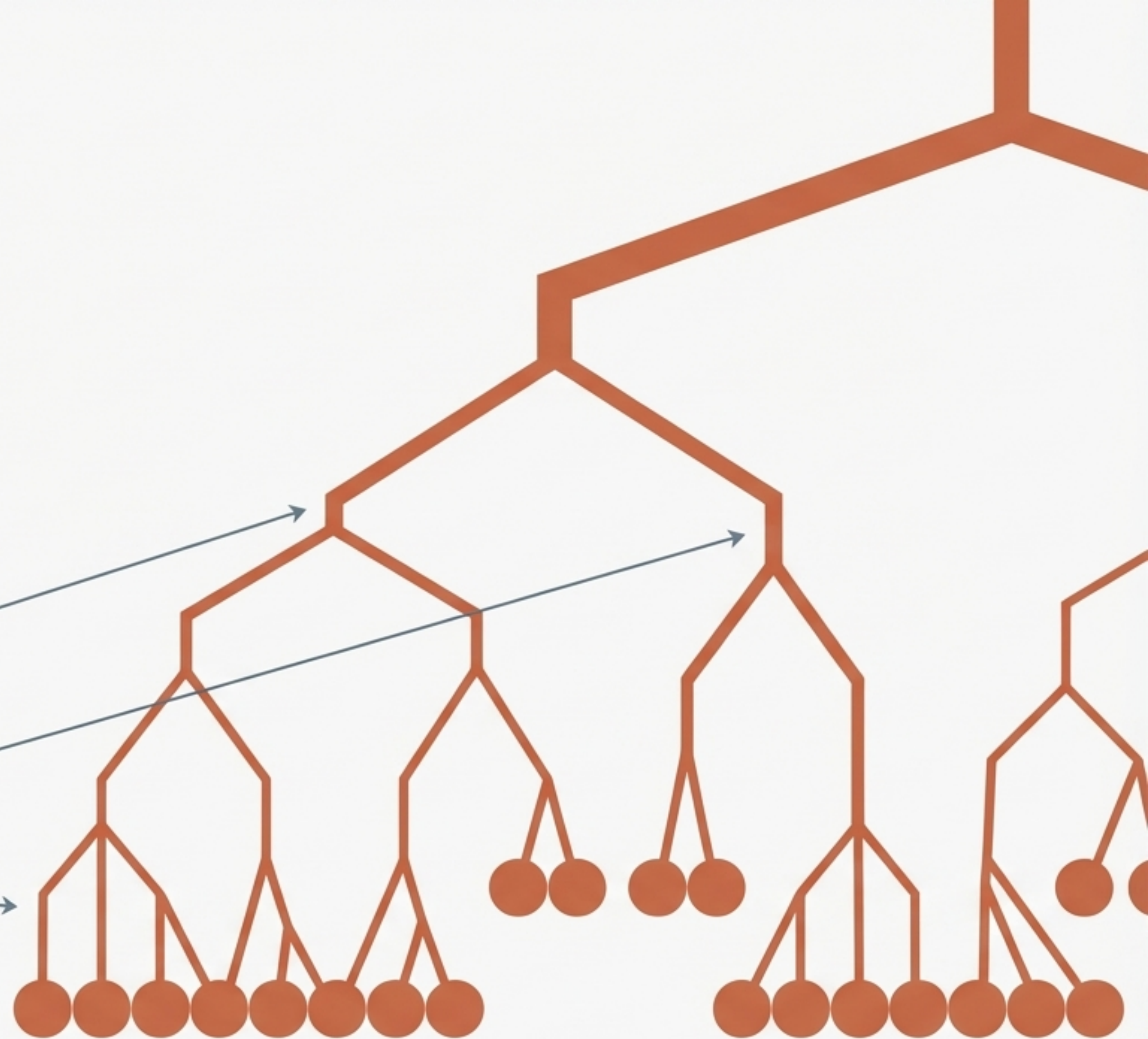


Shaping the inner workings of the trees

Once the forest's size and data allocation are set, we must control the microscopic architecture of the individual decision trees.

Key Parameters:

- `max_depth`
- `min_samples_split`
- `min_samples_leaf`
- `max_leaf_nodes`



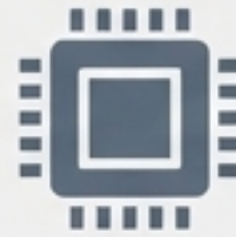
The fundamental goal is pruning

Untuned decision trees will grow indefinitely until they perfectly classify every single training row—memorising noise and causing severe overfitting. The tree parameters act as shears, pruning the branches to maintain generalisation. We achieve this by enforcing minimums (e.g., minimum samples required to split) or using direct pruning metrics like `min_impurity_decrease`.



General system and environment controls

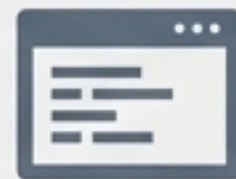
These parameters do not alter the mathematical logic of the Random Forest; they control how your hardware and environment execute the training.



`n_jobs` (in IBM Plex Mono, muted slate blue): Enables parallel processing. Setting this to utilise multiple CPU cores drastically speeds up training.



`random_state` (in IBM Plex Mono, muted slate blue): Locks the randomness. Essential for uniform, reproducible results across different training runs.



`verbose` (in IBM Plex Mono, muted slate blue): Controls the amount of diagnostic text output printed during the training phase.

Specialised operational parameters

For specific data scenarios and advanced training workflows, the algorithm provides targeted levers:



`class_weight`

Crucial for imbalanced datasets. It allows you to assign higher penalties or weights to minority classes.



`warm_start`

Enables batch training. Useful when you acquire new data later and want to add to an existing model without retraining from scratch.

Classifiers vs. Regressors

The hyperparameter architecture for a Random Forest Regressor is nearly identical to the Classifier. The singular distinct difference lies in the splitting criterion.

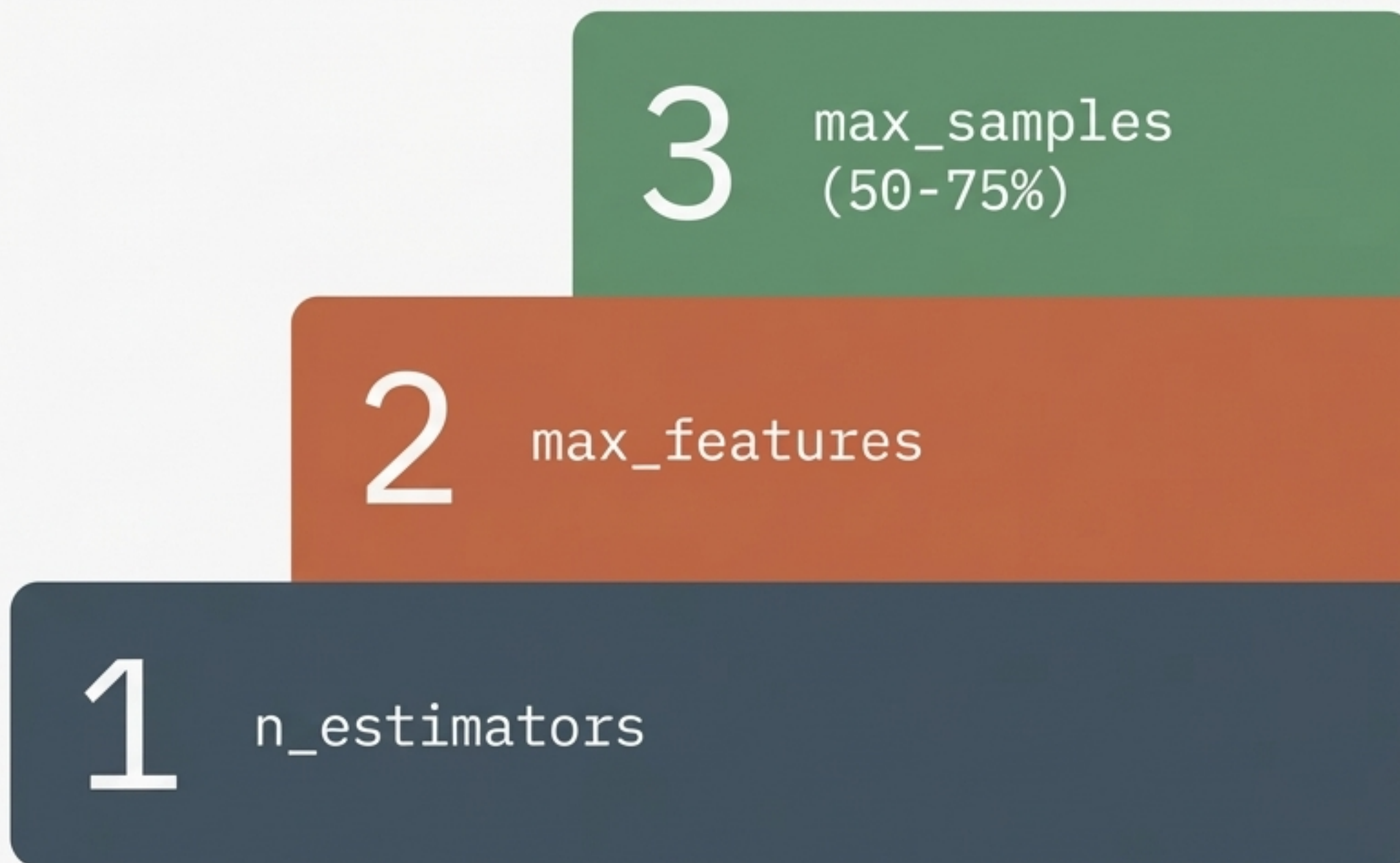
- Classification Criterion: Uses Gini or Entropy to measure node purity.
- Regression Criterion: Uses Mean Squared Error (MSE) or Mean Absolute Error (MAE) to measure variance.

	Classifier	Regressor
n_estimators	✓	✓
max_depth	✓	✓
max_features	✓	✓
min_samples_split	✓	✓
Criterion	Gini / Entropy	MSE / MAE

A practical tuning strategy

Do not assume default values are optimal.
Approach tuning systematically:

1. **Establish the baseline:** Scale up `n_estimators` until the decision boundary smooths out and performance plateaus.
2. **Introduce column randomness:** Experiment with varying percentages of `max_features`.
3. **Dial in row randomness:** Adjust `max_samples` specifically targeting the 50% to 75% range.



Moving from theory to practice

Understanding the 3-part architecture—The Forest, The Trees, and The System—is only the first step. The true impact of `max_samples`, feature selection, and tree pruning is best understood through active experimentation. Apply these levers to real-world datasets and utilise visual tuning tools to observe the shifting decision boundaries in real time.

```
clf = RandomForestClassifier(  
    n_estimators=150,  
    max_samples=0.7 )
```

